

# CIFAR-10 Custom ResNet: Architectural Choices & Design Rationale

Deep Learning Model Architecture Overview | PyTorch Implementation

---

## 1. Data Preprocessing & Synthetic Augmentation Pipeline

To maximize generalization on CIFAR-10's tiny 32x32 pixel images, the input pipeline incorporates a dynamic data transformation strategy. `RandomCrop(32, padding=4)` and `RandomHorizontalFlip()` introduce spatial translation and reflection invariance. Crucially, a `RandomRotation(15)` transformation introduces rotational flexibility, forcing the network to learn geometric orientation invariance. Finally, input tensors are normalized across RGB channels using pre-calculated CIFAR-10 population statistics (mean=[0.4914, 0.4822, 0.4465], std=[0.2023, 0.1994, 0.2010]), centering input data around zero mean and unit variance to stabilize backpropagation gradients.

## 2. Custom Residual Block Design (ResidualBlock)

The fundamental building block employs two 3x3 convolutions with spatial padding of 1, preserving spatial dimensions across stages. To prevent feature map co-adaptation and structural overfitting, a Spatial Dropout layer (`Dropout2d, p=0.2`) is injected directly into the feature mapping pathway. Each convolutional layer is paired with Batch Normalization (`BatchNorm2d`) before ReLU activation; therefore, bias parameters are explicitly omitted (`bias=False`) to eliminate redundant parameters. The skip connection (`out += identity`) forms the core residual path, allowing unimpeded gradient flow during backpropagation and resolving the vanishing gradient problem. When channel capacities increase or spatial strides change, a 1x1 projection convolution (shortcut) aligns tensor shapes for element-wise addition.

## 3. Multi-Stage Stage Progression & Global Pooling

The CustomResNet network is structured into an initial preparation stage (Prep Stage) followed by three hierarchical feature extraction stages configured with [2, 2, 2] blocks:

- **Prep Stage:** 3 to 64 channels (3x3 Conv, Stride 1)
- **Stage 1:** 64 channels, Stride 1 (2 Residual Blocks)
- **Stage 2:** 128 channels, Stride 2 (2 Residual Blocks, downsamples spatial resolution by half)
- **Stage 3:** 256 channels, Stride 2 (2 Residual Blocks, downsamples spatial resolution by half)

Following Stage 3, `AdaptiveAvgPool2d((1, 1))` collapses spatial dimensions into 1x1 feature maps per channel. This replaces heavy fully connected layers with a single `Linear(256, 10)` classifier, drastically reducing parameter count while remaining spatially invariant.

## 4. Optimization, Regularization, and Learning Rate Scheduling

Model parameters are optimized using Stochastic Gradient Descent (SGD) with a high momentum factor of 0.9 to smooth gradient direction trajectories. L2 Regularization (Weight Decay =  $1e-3$ ) is enforced to penalize excessively large weights and preserve decision boundary smoothness. Training is governed by a `MultiStepLR` scheduler starting at an initial learning rate of 0.15, applying aggressive 90% drops (`gamma=0.1`) at pre-defined milestones (Epochs 10, 20, and 25). This multi-stage deceleration allows broad global gradient exploration early on, followed by fine-grained precision convergence in the final epochs.

## 5. Execution Safety & Performance Engineering

The training execution pipeline includes Windows-specific multiprocessing guards (if `__name__ == '__main__':`) to ensure safety under Windows process spawning. Data loaders utilize pinned host memory (`pin_memory=True`) for accelerated host-to-device transfers, while non-blocking transfers (`to(device, non_blocking=True)`) overlap CPU data staging with GPU tensor processing.